Is Coding *Dead?*

Java in the AI Era.

What's real, what's hype, and where you fit.

Is coding dead?

Two kinds of noise drown out a calmer, more useful truth.

“Coding is dead”

Doomers say AI ends the job. Reality: the floor rose and boilerplate got cheap. But software is more than typing, and demand for people who understand systems isn't shrinking.

Doomer

Snake oil & YouTube-ism

Influencers say prompts replace engineers. They sell courses, not outcomes. Tune out the thumbnails; watch what actually ships.

Snake oil

AI is a real productivity shift not a replacement.

The winners pair it with real engineering. Java already runs the enterprise systems that AI needs to plug into.

The **value of AI** compounds with the **value of Java.**

Why Java matters in the AI era

AI doesn't run on notebooks in production. It runs where business applications already live. That's Java.

WHERE PYTHON FITS

Research, model training, data science, notebooks, the cutting edge of ML tooling.

WHERE JAVA FITS

Production & enterprise integration, concurrency, scale, type safety, ***security***, systems of record.

THE AI STACK FOR JAVA

Your app
Quarkus · Spring

→

Orchestration
LangChain4j · Spring AI

→

Models
LLM API · local

→

Vector DB
embeddings

→

Your data
systems of record

AI concepts you must know

Start with the two fundamental ideas. How a model behaves and how to ground it in your data.

THE MODEL

How LLMs work

Non-deterministic by design. The same prompt can give different answers. You don't program it, you steer it.

Non-determinism | Prompting | Context

YOUR DATA

RAG

Retrieval-Augmented Generation (RAG) grounds answers in your own content, so the model cites what you know, not what it guessed.

Similarity | Embeddings | Vector DBs

RAG FLOW

Question → Embed → Vector DB → Retrieve → LLM + context → Grounded answer

Agents

An LLM that can take actions. Looping until a goal is met turns answers into work.

WHAT MAKES IT AN AGENT

Give the model tools and a goal, and let it decide the steps. It calls your functions to get things done.

Loop **Tools / function calling** **Memory**

WHEN TO REACH FOR ONE

Multi-step tasks that touch real systems… Query a DB, call an API, file a ticket. For simpler Q&A over docs, modern RAG is enough.

THE AGENT LOOP

Plan → **Act · call a tool** → **Observe**

↻ repeat until the goal is met tools: query DB · hit API · run code

Your AI toolkit

Some suggestions for building generative AI apps without leaving the JVM.

01	**AI assistants**	In-IDE pair programming for everyday coding, refactors and tests.
02	**Local LLMs**	Run models on your own machine… private, offline, zero per-token cost. (Ollama)
03	**LangChain4j**[1]	The orchestration layer: chat, RAG, tools and agents, idiomatic Java.
04	**Quarkus LC4j & Spring AI**	First-class framework integration — config, DI and observability built in.
05	**Embabel**	Open-source agent framework based on Goal-Oriented Action Planning (GOAP)

1 

Predictive AI on the JVM

Not everything is GenAI and LLMs. Classic machine learning still ships and it's all here in Java.

DeepLearning4j (Eclipse Deeplearning4j)

Distributed deep learning & neural nets, built for the JVM and big-data stacks.

DeepNetts

Pure-Java deep learning with a Java-native API. Very easy to embed straight into apps.

Weka

The classic ML workbench: classification, regression and clustering, out of the box.

Mahout

Scalable, distributed linear algebra and machine learning for large datasets.

Reach for these when the job is forecasting · classification · anomaly detection. No LLM required.

Where to focus your growth

The model writes code. Your edge is everything around the code.

THE COMPOUNDING SKILL

Critical thinking vs. code generation

Generating code is cheap. Knowing whether it's correct, safe and right for the problem is the job.

Heterogeneous system design

Java, Python, models & data working as one system.

Collaboration

Working with AI, and with people, as a force multiplier.

Security & privacy

Where data goes, prompt injection, what stays on-prem.

Ops & risk

Costs, observability, explainability, risk management.

Getting Started

Before you jump into the deep end of the pool, learn how to swim

A Core Technical Skills

Algebra, statistics, ML concepts, neural networks, software engineering basics, data structures/algorithms, OS, networking, databases (SQL, MySQL), Java ecosystem (JVM, concurrency, performance tuning, cloud-native Java)

B Modern Dev Skills

Learn production deployment, AI engineering (prompt engineering, RAG, agent workflows, evaluation/testing), cloud platforms, Docker, Kubernetes, AI-assisted dev tools, version control, secure coding, API design

C Hands-On Practice & Open Source

Building applications, working on real-world projects (problem-solving, testing, documentation, monitoring), open source contributions (reading codebases, PRs, code reviews, git, CI/CD, issue trackers), building AI tool integrations, participate in hackathons/contests

D Career Readiness & Community

Building a portfolio, engaging with AI/tech communities, internships, communication skills(!), understanding business value/privacy/UX, and interview prep (coding, system design, discussing projects in depth)

Practical projects for your CV

Small, shippable, and proof that you can build applications and not just prompt.

A Simple RAG

Index your own docs and answer questions grounded in them. The "hello world" of useful AI.

B Private chatbot

A local LLM behind a Java service. Where nothing leaves your machine. Great for sensitive data.

C Agent with Java tools

Let the model call your functions e.g., query a DB, hit an API via tool/function calling. Understand the agent loop.

D Streaming AI

Stream tokens to the UI as they generate. Yields the responsive feel users now expect.

More Tips for New Developers

Consider these suggestions

- Identify which problems are worth addressing. AI agents are good at solving well-specified problems.
- Prioritize your time and build professional relationships. This will increase your opportunities.
- Increase your domain knowledge. This will help you guide AI to better solutions.
- Select ambitious problems to solve. AI can solve the easy ones.
- Look for situations where problems outnumber the team. This will force you to learn and accept responsibility.
- If possible, optimize for career opportunities. Compensation will follow.
- Be a contributor to the meat of a project.
- Develop a researcher’s mentality. Be curious. Experiment and measure results.
- You typically have to fail at a task before you succeed. Failure is part of the learning process.
- Provide excellence and extreme attention to detail.

Coding isn't dead. *It leveled up.*

The floor rose, the job moved up the stack.
Java sits exactly where enterprise AI has to run.